CYA-2: Quantum Computing Report -- Grover's Algorithm

- Author: Runlin Zhao

This report primarily provides a brief summary of Grover's Algorithm. First, it outlines the main implementation steps; second, it explains the specific meaning of each step from the perspective of geometric interpretation and elaborates on the fundamental concept of the algorithm -- amplitude amplification. Next, it provides a performance analysis of the algorithm. Finally, simple practical examples are presented.

References

- Quantum Computation and Quantum Information - Nielsen
- Grover's Algorithm - Qiskit
- Introduction to Quantum Computing and Quantum Hardware — Youtube
- Solving Satisfiability Problems using Grover's Algorithm - Qiskit
- Amplitude Amplification - PyQPanda
- Theory of Grover's search algorithm - Microsoft

Main Steps

Unstructured Search Problem

Suppose we are given a massive list of N items. Within this list, there is one item that possesses a unique property, and our goal is to locate it. Let us denote this target item as w. For instance, imagine this item is green, while all other items are blue.

To find this unique green item using classical search methods, one would have to check an average of $\frac{N}{2}$ times, and in the worst-case scenario, N times. However, if we have a quantum computer and use Grover's algorithm, we only need roughly $\sqrt{N}$ searches. Obviously, the larger the list of items, the greater the advantage of a quantum computer over a classical one.

A specific case of the search problem can be simply represented as a function f with input x, where $f(x) = 1$ if x is a solution to the search problem, and $f(x) = 0$ otherwise.

Constructing an oracle

To describe the algorithm, I will proceed from the specific components to the overall structure. Therefore, I will first present the construction method of a small component in the algorithm: the oracle.

The role of the oracle is to recognize the solution to the search problem. This recognition is indicated by a quantum bit (qubit): if $f(x) = 1$, the qubit is flipped; otherwise, it remains unchanged. In the computational basis, we can define it

as follows:

$$|x\rangle|q\rangle \xrightarrow{O} |x\rangle|q \bigoplus f(x)\rangle$$

In fact, the oracle simply applies a -1 phase to the solution, which we can write as:

$$|x\rangle \xrightarrow{O} (-1)^{f(x)}|x\rangle$$

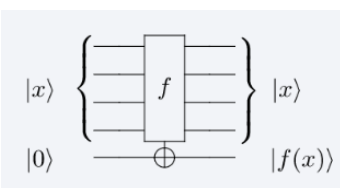
Therefore, this oracle is a unitary operator, and we can write it as a unitary matrix:

$$U_w|x\rangle = (-1)^{f(x)}|x\rangle$$

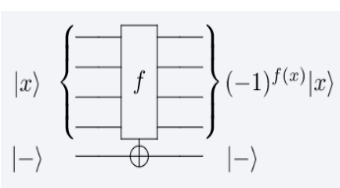
The matrix is as follows:

$$U_w = \begin{pmatrix} (-1)^{f(0)} & 0 & \cdots & 0 \\ 0 & (-1)^{f(1)} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & (-1)^{f(2^n-1)} \end{pmatrix}$$

How do we construct this circuit? Recall the circuit we used in the Deutsch-Jozsa algorithm:



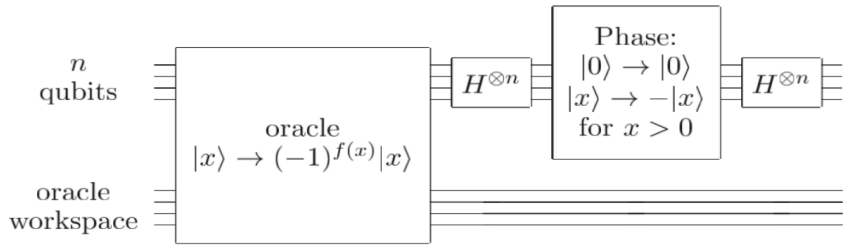
We can slightly modify it by changing $|0\rangle$ to $|-\rangle$ to obtain the required oracle.



The Grover Operator

After obtaining the basic oracle, we can use it to construct a Grover operator. In the overall circuit, this Grover operator needs to be applied multiple times.

- The Grover operator can be described in the following four steps:
  - Apply the oracle once;
  - Apply the Hadamard transform $H^{\otimes n}$ once;
  - Perform a conditional phase shift on the computer, where every computational basis state except $|0\rangle$ receives a phase shift of -1: $|x\rangle \longrightarrow -(-1)^{\delta_{x0}}|x\rangle$; (where δ is the Kronecker delta function)
  - Apply the Hadamard transform $H^{\otimes n}$ once more.
- The circuit of the Grover operator is shown below:



We can perform some simple derivations to write the Grover operator as a unitary operator. The two Hadamard transforms are obvious; we only need to represent the third step (conditional phase shift) as a unitary operator.

Deriving the unitary operator for the third step's phase shift: Through a simple check, the unitary operator that satisfies the condition is $2|0\rangle\langle 0| - I$. We can verify this by substituting:

$$(2|0\rangle\langle 0| - I)|0\rangle = 2|0\rangle - |0\rangle = |0\rangle$$
$$(2|0\rangle\langle 0| - I)|x\rangle = |0\rangle - |x\rangle = -|x\rangle$$

It clearly holds for both cases.

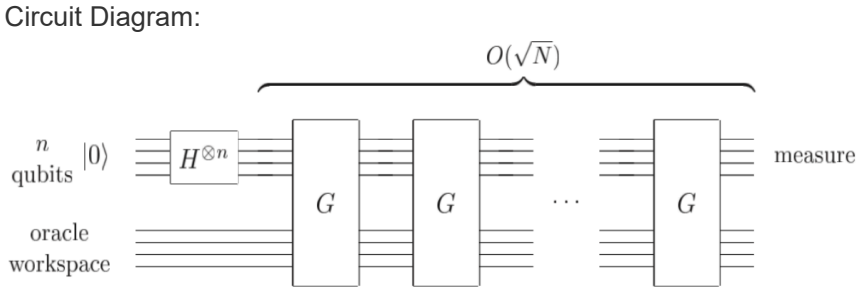
Let the initial state $|0\rangle^{\otimes n}$ go through a Hadamard transform to become a uniform superposition state $|\psi\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1} x$. Then, integrating the expression from the third step with the second and fourth steps, we get:

$$H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n} = 2|\psi\rangle\langle\psi| - I$$

Therefore, the entire Grover operator can be written as $G = (2|\psi\rangle\langle\psi| - I)O$. Note the order here: the operator applied first is written on the right.

Overall Algorithm Steps

- Algorithm: Quantum Search
- Inputs:
  - A black-box oracle O, such that $|x\rangle|q\rangle \xrightarrow{O} |x\rangle|q \bigoplus f(x)\rangle$, where $f(x_0) = 1$, and $f(x) = 0$ for all other cases $0 \leq x < 2^n$.
  - $n + 1$ qubits initialized to $|0\rangle^{\otimes n}$.
- Output: x_0
- Runtime: $O(\sqrt{2^n})$ operations. Success probability $O(1)$.
- Circuit Diagram:



- Procedure:
  - Initial state: $|0\rangle^{\otimes n}|0\rangle$
  - Apply $H^{\otimes n}$ to the first n qubits, and HX to the last qubit, resulting in:

$$\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|x\rangle[\frac{|0\rangle-|1\rangle}{\sqrt{2}}]$$

- Apply the Grover operator approximately $R \approx [\pi\sqrt{2^n}]/4$ times, resulting in:

$$[(2|\psi\rangle|\psi\langle-I)O]^R\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|x\rangle[\frac{|0\rangle-|1\rangle}{\sqrt{2}}] \approx |x\rangle[\frac{|0\rangle-|1\rangle}{\sqrt{2}}]$$

- Measure the first n qubits to obtain x_0.

Geometric Interpretation

From the algorithm steps above, we can see that the first two steps are initialization, and the last step is obtaining the result. The most critical part is the third step, which applies the Grover operator approximately $R \approx [\pi\sqrt{2^n}]/4$ times. What is the role of the Grover operator? Why does it need to be executed R times? It is difficult to understand intuitively from equations alone, but it becomes very clear when viewed geometrically. Therefore, this section explains the geometric meaning of the algorithm.

The Grover Operator as a Rotation

If we observe the two-dimensional space spanned by the initial state $|\psi\rangle$ and the uniform superposition state of the search problem's solutions, we will find that the Grover operator can be seen as a rotation transformation in this 2D space. The derivation is as follows:

Assume there are N items in total, of which M are solutions. Let $\sum_x{}'$ denote the sum over all x that are solutions, and $\sum_x{}''$ denote the sum over all non-solutions. We can define normalized states:

$$|\alpha\rangle = \frac{1}{\sqrt{N-M}}\sum_x{}''|x\rangle \tag{1}$$

$$|\beta\rangle = \frac{1}{\sqrt{M}}\sum_x{}'|x\rangle \tag{2}$$

Thus, the initial state can be re-expressed as:

$$|\psi\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}x \tag{3}$$

$$= \sqrt{\frac{N-M}{N}}|\alpha\rangle + \sqrt{\frac{M}{N}}|\beta\rangle \tag{4}$$

Clearly, the sum of the squares of the coefficients is 1, so the initial state can be represented as a unit vector in the space spanned by $|\alpha\rangle$ and $|\beta\rangle$. If we let the angle between the initial state and $|\alpha\rangle$ be $\frac{\theta}{2}$ (set this way to make the subsequent geometric derivations simpler, as both operations are reflections), the initial state can be written as $|\psi\rangle = \cos\frac{\theta}{2}|\alpha\rangle + \sin\frac{\theta}{2}|\beta\rangle$.

Next, apply the Grover operator to the initial state.

First is O. According to the definition of the oracle, it's clear that:

$$
\begin{aligned}
O(|\psi\rangle) &= O(\cos\frac{\theta}{2}|\alpha\rangle + \sin\frac{\theta}{2}|\beta\rangle) && (5)\\
&= \cos\frac{\theta}{2}O(|\alpha\rangle) + \sin\frac{\theta}{2}O(|\beta\rangle) && (6)\\
&= \cos\frac{\theta}{2}(-1)^{f(\alpha)}|\alpha\rangle + \sin\frac{\theta}{2}(-1)^{f(\beta)}|\beta\rangle && (7)\\
&= \cos\frac{\theta}{2}|\alpha\rangle - \sin\frac{\theta}{2}|\beta\rangle && (8)
\end{aligned}
$$

So the oracle operation is essentially a reflection of the initial state about the $|\alpha\rangle$ axis in the plane spanned by $|\alpha\rangle$ and $|\beta\rangle$. Thus, it can be written as $O = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$.

Second is $2|\psi\rangle\langle\psi| - I$.

Substituting the initial state yields:

$$
\begin{aligned}
2|\psi\rangle\langle\psi| - I &= 2\begin{pmatrix} \cos\frac{\theta}{2} \\ \sin\frac{\theta}{2} \end{pmatrix}\begin{pmatrix} \cos\frac{\theta}{2} & \sin\frac{\theta}{2} \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} && (9)\\
&= \begin{pmatrix} \cos\theta & \sin\theta \\ \sin\theta & -\cos\theta \end{pmatrix} && (10)
\end{aligned}
$$

Evidently, this operation is also a reflection, specifically a reflection about the vector making an angle $\frac{\theta}{2}$.

Combining the two gives the Grover operator $G = \left(\begin{smallmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{smallmatrix}\right)$. Thus, G is a rotation transformation in the two-dimensional space spanned by $|\alpha\rangle$ and $|\beta\rangle$. The derivation is complete.

Arbitrary Iterations on the Initial State

So what do we get when we apply the Grover operator k times to the initial state?

Substituting for one application, we get:

$$
G|\psi\rangle = \begin{pmatrix} \cos(\frac{\theta}{2} + \theta) \\ \sin(\frac{\theta}{2} + \theta) \end{pmatrix} = \cos\frac{3\theta}{2}|\alpha\rangle + \sin\frac{3\theta}{2}|\beta\rangle
$$

So the result of an arbitrary k applications is $G^k|\psi\rangle = \cos(\frac{2k+1}{2}\theta)|\alpha\rangle + \sin(\frac{2k+1}{2}\theta)|\beta\rangle$.

Algorithm Basic Idea (Amplitude Amplification)

The fundamental idea of Grover's algorithm actually stems from amplitude amplification, making it a special case of the amplitude amplification algorithm.

The amplitude amplification algorithm can be described as follows:

First, let $\chi: \mathbb{Z} \to \{0, 1\}$ be a boolean function. An input $z \in \mathbb{Z}$ satisfying $\chi(z) = 1$ is considered a solution. Second, assume we have an algorithm to verify whether a given input is a solution, denoted as an oracle. Written as a unitary operator, this is $|z\rangle \xrightarrow{O_\chi} (-1)^{\chi(z)}|z\rangle$. Next, assume we have an algorithm $\mathcal{A}$ that, when applied to the initial state, has a probability p of finding a solution, and does not involve intermediate measurements. Then, assume the basis state for solutions is $|G\rangle$, and the basis state for non-solutions is $|B\rangle$, and these two are orthogonal.

Using classical methods, we would need to repeat $\mathcal{A}$ roughly $1/p$ times to find a solution. However, using the amplitude amplification algorithm, we only need to run $\mathcal{A}$ and $\mathcal{A}^{-1}$ roughly $O(1/\sqrt{p})$ times. The algorithm steps are:

- Step 1: Set the initial state $|U\rangle = \mathcal{A}|0\rangle$.
- Step 2: Repeat the following steps $O(1/\sqrt{p})$ times:
  - Reflect about $|B\rangle$, which is essentially applying O_χ.
  - Reflect about $|U\rangle$, which is essentially applying $\mathcal{A}\mathcal{R}\mathcal{A}^{-1}$, where $\mathcal{R}$ is a specific operator that satisfies the reflection requirement.
- Step 3: Measure the first register and check if it is a solution.

Evidently, amplitude amplification works by magnifying the amplitude of a specific basis state within a uniform superposition, thereby obtaining a desired target quantum state that yields the requested solution with high probability upon measurement.

Any algorithm that has a non-trivial probability of finding a solution can utilize amplitude amplification. Grover's algorithm is clearly a special case where the Hadamard transform can be viewed as an algorithm that has a probability $p = M/N$ of finding a solution to a search problem of size N with M solutions. Thus, it corresponds to the case where $O_\chi = O_{x,\pm}$ and $\mathcal{A} = H^{\otimes n}$.

Performance Analysis

How Many Iterations Are Needed?

We previously mentioned that the Grover operator needs to be applied a certain number of times. Why is this specific number of iterations sufficient? The derivation is as follows.

Suppose the initial uniform superposition state after the Hadamard gates is $|\text{all}\rangle$, the basis state for solutions is $|\text{good}\rangle$, and the basis state for non-solutions is $|\text{bad}\rangle$. The problem size is N, the number of solutions is M, and the angle is $\frac{\theta}{2}$.

It must be clarified that Grover's algorithm should only be used when $M \ll N$. Otherwise, if, for example, $M = \frac{N}{4}$, then its performance is even worse than a classical algorithm. Furthermore, once $M > \frac{N}{2}$, from a geometric perspective, a single application of the Grover operator would immediately rotate the state out of the first quadrant.

Returning to the main point. The initial angle is $\frac{\theta}{2} = \arccos(\langle \text{all}|\text{bad}\rangle) = \arccos(\sqrt{\frac{N-M}{N}})$.

Let the angle between $|\text{good}\rangle$ and the state register after k iterations be $\gamma(k)$, then $\gamma(k) = \frac{\pi}{2} - (2k+1)\frac{\theta}{2}$.

Therefore, the probability of success is:

$$P(\text{success}) = \cos^2(\gamma(k)) = \sin^2\left[(2k+1)\arccos\left(\sqrt{\frac{N-M}{N}}\right)\right]$$

The probability is maximized for the first time when:

$$\frac{\pi}{2} = (2k_{\text{optimal}} + 1)\arccos\left(\sqrt{\frac{N-M}{N}}\right)$$

Solving this gives:

$$k_{\text{optimal}} = \frac{\pi}{4}\sqrt{\frac{N}{M}} - \frac{1}{2} - O\left(\sqrt{\frac{M}{N}}\right)$$

So the optimal number of iterations is roughly $\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} - \frac{1}{2} \rfloor$.

Can the Number of Iterations Be Fewer?

Is there a quantum algorithm that can complete the search task with fewer than $\Omega(\sqrt{N})$ queries?

The answer is no. In 1997, Bennett et al. proved in a paper that Grover's algorithm is asymptotically optimal.

Additionally, Nielsen provides a detailed proof in Section 6.6 of his book, so I will not reiterate it here.

Code Implementation

The code implementation here primarily references the official Qiskit documentation.

Environment Setup

On our personal devices, we can use classical computers to simulate quantum computers. There are already several tools available for this, and Qiskit is the best one I have used so far.

If you want to try a real quantum computer, you can visit the Qiskit website, open IBM Quantum Lab, and register for an account.

The process for setting up the environment to simulate a quantum computer on a local device is as follows:

First, download and install Anaconda. Following the official Qiskit documentation, it's recommended to configure a virtual environment and then install packages directly using pip.

Next, after setting up the environment, import the newly created virtual environment into Jupyter Notebook. Activate the virtual environment, install the necessary package using conda install ipykernel, and then import the environment into Jupyter Notebook using the following command:

```
python -m ipykernel install --user --name MyQiskitTest --display-name "Qiskit"
```

Additionally, you need to install the qiskit-textbook. Follow the installation instructions on its GitHub page.

Once everything is ready, the environment setup is complete.

Use the following command to verify the current environment:

```
import qiskit.tools.jupyter
%qiskit_version_table
```

Circuit Design

How do we design the circuit?

For the oracle part, we can implement it using a controlled-Z gate.

For the amplitude amplification part, the two H gates are straightforward. The main focus is the intermediate step, which can be represented as:

$$U_s = H^{\otimes n} U_0 H^{\otimes n}$$

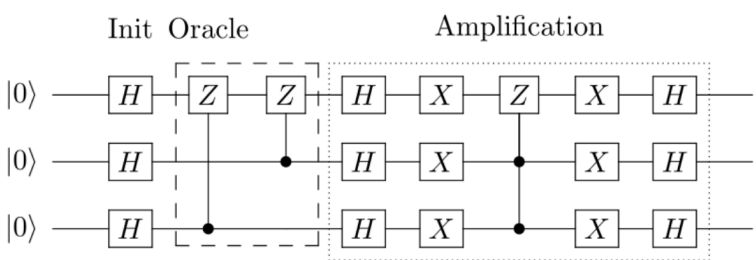
How do we apply a -1 phase to everything except 0? We can do it as follows:

$$U_s = -H^{\otimes n} U_0 H^{\otimes n} = H^{\otimes n} X^{\otimes n} (MCZ) X^{\otimes n} H^{\otimes n}$$

Where MCZ is:

$$MCZ = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \end{bmatrix} \leftarrow \text{Add negative phase to } |11\ldots1\rangle$$

So, using three qubits as an example, if the marked states are $|101\rangle$ and $|110\rangle$, the overall circuit looks like the following diagram:



Code Implementation

The official documentation provides examples for two and three qubits, as well as instructions for constructing a generic circuit. The official YouTube tutorial video also provides an arbitrary-qubit version.

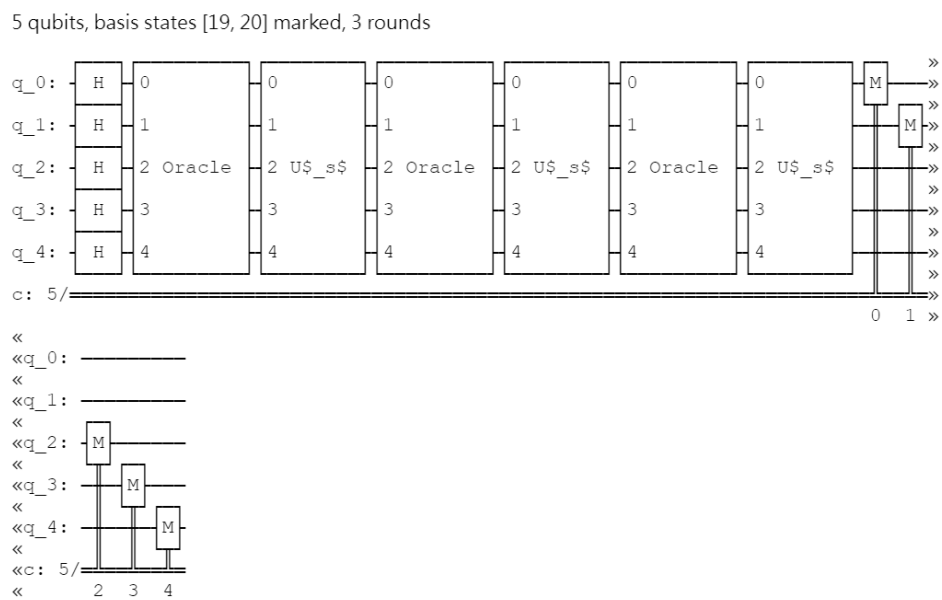
Based on the documentation, the code to implement the circuit is as follows:

```python
from qiskit.quantum_info import Operator
from qiskit import QuantumCircuit
import numpy as np

def phase_oracle(n, indices_to_mark, name = 'Oracle'):

    qc = QuantumCircuit(n, name=name)
    oracle_matrix = np.identity(2**n)

    for index_to_mark in indices_to_mark:
        oracle_matrix[index_to_mark, index_to_mark] = -1

    qc.unitary(Operator(oracle_matrix), range(n))

    return qc


def diffuser(nqubits):

    qc = QuantumCircuit(nqubits)

    for qubit in range(nqubits):
        qc.h(qubit)

    for qubit in range(nqubits):
        qc.x(qubit)

    qc.h(nqubits-1)
    qc.mct(list(range(nqubits-1)), nqubits-1)  # multi-controlled-toffoli
    qc.h(nqubits-1)

    for qubit in range(nqubits):
        qc.x(qubit)

    for qubit in range(nqubits):
        qc.h(qubit)

    U_s = qc.to_gate()
    U_s.name = "U$_s$"

    return U_s


def Grover(n, indices_of_marked_elements):

    # Create a quantum circuit on n qubits
    qc = QuantumCircuit(n, n)

    # Determine r
    r = int(np.floor(np.pi/4*np.sqrt(2**n/len(indices_of_marked_elements))))
```

```
    print(f'{n} qubits, basis states {indices_of_marked_elements} marked, {r} rounds')

    # step 1: apply Hadamard gates on all qubits
    qc.h(range(n))

    # step 2: apply r rounds of the phase oracle and the diffuser
    for _ in range(r):
        qc.append(phase_oracle(n, indices_of_marked_elements), range(n))
        qc.append(diffuser(n), range(n))

    # step 3: measure all qubits
    qc.measure(range(n), range(n))

    return qc

mycircuit = Grover(5, [19, 20])
mycircuit.draw()
```

The output circuit is shown below:

Out[4]:

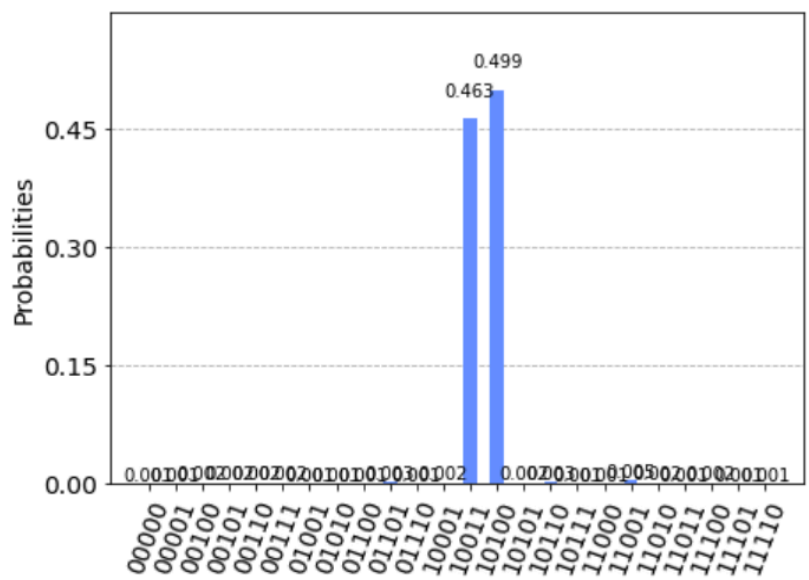


This is a five-qubit circuit where the marked numbers are 19 and 20. To plot the measurement results, use the following code:

```
from qiskit import Aer, execute
from qiskit.visualization import plot_histogram


simulator = Aer.get_backend('qasm_simulator')
counts = execute(mycircuit, backend=simulator, shots=1000).result().get_counts(mycircuit)

plot_histogram(counts)
```

The result after running is as follows, indicating a clear success:

Out[5]:



Simple Application I: Sudoku Model

The Problem

Having discussed Grover's algorithm, let's provide a concrete example.

For instance, when solving a Sudoku puzzle, instead of a brute-force search (although there are better classical algorithms, this is just an example), we can use Grover's algorithm to accelerate the process.

Due to the limitation of qubit counts, we will only discuss a simplified Sudoku model.

This Sudoku model is a 2×2 grid where all elements are either 0 or 1. There are two simple rules: no identical elements are allowed in any row, and no identical elements are allowed in any column. We need to find a solution that satisfies these conditions.

$$\begin{bmatrix} v_0 & v_1 \\ v_2 & v_3 \end{bmatrix}$$

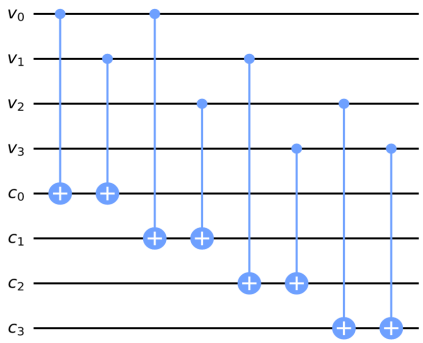
Analysis

If we were to use a classical computer for a brute-force search, assuming the four numbers are $v_0,\ v_1,\ v_2,\ v_3$, it would equate to traversing 2^4 possibilities. Written in binary, a valid solution must satisfy:

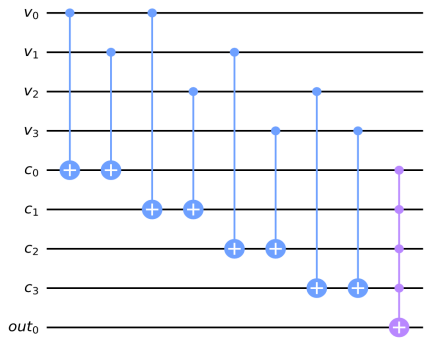
$$\begin{aligned} v_0 &\neq v_1 \\ v_2 &\neq v_3 \\ v_0 &\neq v_2 \\ v_1 &\neq v_3 \end{aligned}$$

If we want to use a quantum computer and Grover's algorithm, comparing it to the code in the previous section, we clearly only need to modify the oracle part. We just need the oracle to determine if a set of solutions meets the criteria (i.e., the four inequalities listed above).

Naturally, we can think of using CNOT gates. Construct the circuit as shown below:



As long as one of the four inequalities is not satisfied, a certain c_i will inevitably be 0. If all are satisfied, we can use a multi-controlled-Toffoli gate to output a 1.

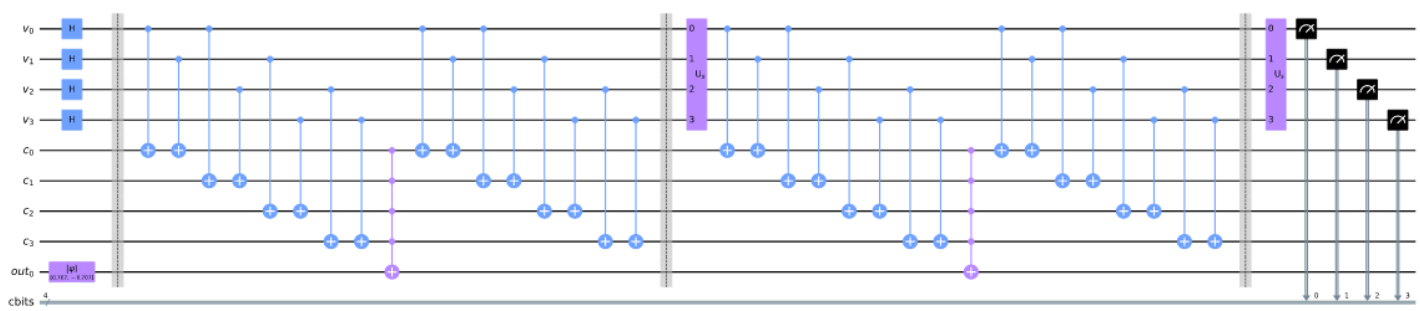


However, note that after one iteration, c_i is no longer its initial 0. Therefore, we need to uncompute the oracle part (excluding the Toffoli gate) to reset c_i back to 0 for the next search iteration.

The steps after the oracle remain the same, which is the amplitude amplification described earlier.

Circuit

The entire circuit is as follows:



Code

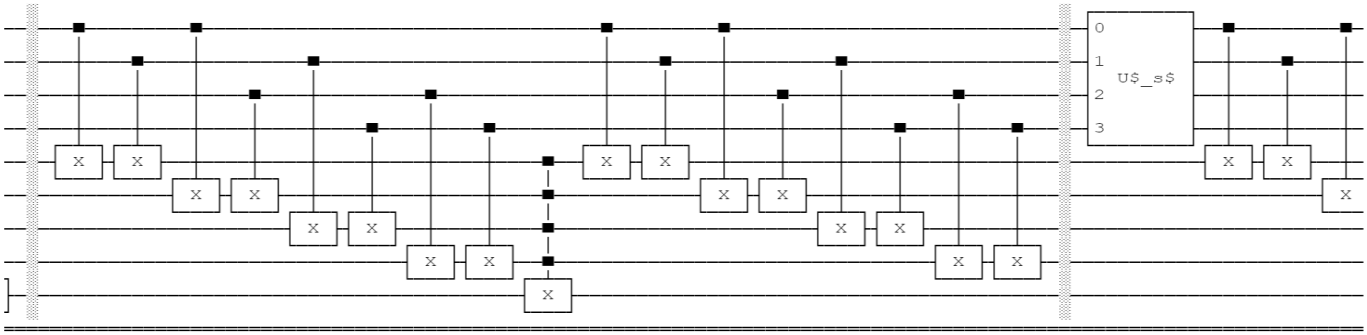
The code is as follows:

```python
#initialization
import matplotlib.pyplot as plt
import numpy as np

# importing Qiskit
from qiskit import IBMQ, Aer, assemble, transpile
from qiskit import QuantumCircuit, ClassicalRegister, QuantumRegister
from qiskit.providers.ibmq import least_busy

# import basic plot tools
from qiskit.visualization import plot_histogram


def XOR(qc, a, b, output):
    qc.cx(a, output)
    qc.cx(b, output)


def diffuser(nqubits):

    qc = QuantumCircuit(nqubits)

    for qubit in range(nqubits):
        qc.h(qubit)

    for qubit in range(nqubits):
        qc.x(qubit)

    qc.h(nqubits-1)
    qc.mct(list(range(nqubits-1)), nqubits-1)  # multi-controlled-toffoli
    qc.h(nqubits-1)

    for qubit in range(nqubits):
        qc.x(qubit)

    for qubit in range(nqubits):
        qc.h(qubit)

    U_s = qc.to_gate()
    U_s.name = "U$_s$"

    return U_s


def sudoku_oracle(qc, clause_list, clause_qubits):
    # Compute clauses
    i = 0
    for clause in clause_list:
        XOR(qc, clause[0], clause[1], clause_qubits[i])
        i += 1
```

```
    # Flip 'output' bit if all clauses are satisfied
    qc.mct(clause_qubits, output_qubit)

    # Uncompute clauses to reset clause-checking bits to 0
    i = 0
    for clause in clause_list:
        XOR(qc, clause[0], clause[1], clause_qubits[i])
        i += 1


clause_list = [[0,1], [0,2], [1,3], [2,3]]

var_qubits = QuantumRegister(4, name='v')
clause_qubits = QuantumRegister(4, name='c')
output_qubit = QuantumRegister(1, name='out')
cbits = ClassicalRegister(4, name='cbits')
qc = QuantumCircuit(var_qubits, clause_qubits, output_qubit, cbits)

# Initialize 'out0' in state |->
qc.initialize([1, -1]/np.sqrt(2), output_qubit)

# Initialize qubits in state |s>
qc.h(var_qubits)
qc.barrier()  # for visual separation

## First Iteration
# Apply our oracle
sudoku_oracle(qc, clause_list, clause_qubits)
qc.barrier()  # for visual separation
# Apply our diffuser
qc.append(diffuser(4), [0,1,2,3])

## Second Iteration
sudoku_oracle(qc, clause_list, clause_qubits)
qc.barrier()  # for visual separation
# Apply our diffuser
qc.append(diffuser(4), [0,1,2,3])

# Measure the variable qubits
qc.measure(var_qubits, cbits)

qc.draw(fold=-1)
```

A portion of the output circuit is shown below:

Out[3]:

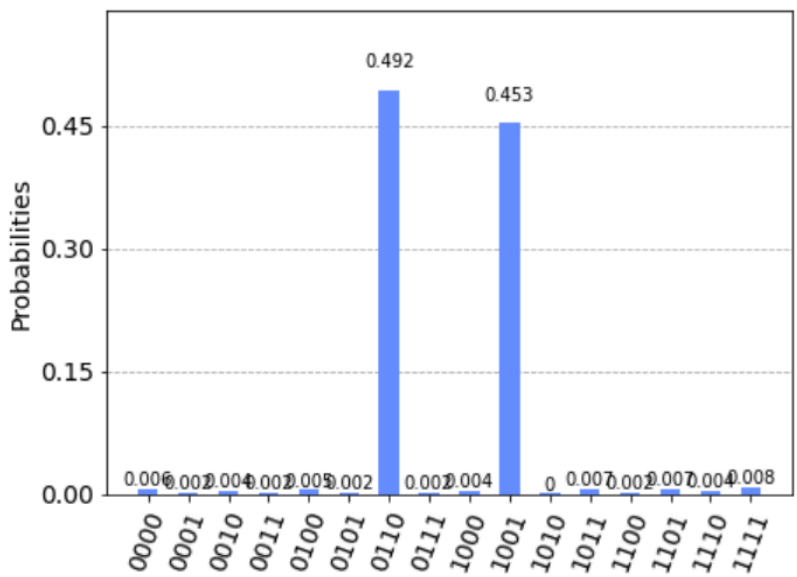


The code to plot the histogram based on the results is:

```
# Simulate and plot results
aer_simulator = Aer.get_backend('aer_simulator')
transpiled_qc = transpile(qc, aer_simulator)
qobj = assemble(transpiled_qc)
result = aer_simulator.run(qobj).result()
plot_histogram(result.get_counts())
```

The result is as follows:

Out[5]:



Thus, the states that satisfy the conditions are $|0110\rangle$ and $|1001\rangle$.

Simple Application II: Real Sudoku

The Sudoku model provided in the Qiskit documentation is merely a simplified example and is still quite different from real-world Sudoku problems.

Regarding real-world Sudoku problems, this GitHub Project provides an implementation of a basic 4×4 Sudoku. The underlying principle is the same as described in the Sudoku model section.

Higher-order Sudoku puzzles can be solved similarly, but the current challenge lies in the need for a significantly larger number of qubits.

Simple Application III: SAT Problem

SAT, or the Boolean satisfiability problem, asks whether a given Boolean expression can be evaluated to TRUE by finding a valid assignment of truth values to its variables.

There is extensive theory on this topic, which we will not expand on here. We are merely using it as an example.

If we attempt to use Grover's algorithm to search for a solution to this problem, it is demonstrably feasible.

For example, consider the following expression:

$$f(v_1, v_2, v_3) = (\neg v_1 \vee \neg v_2 \vee \neg v_3) \wedge (v_1 \vee \neg v_2 \vee v_3) \wedge (v_1 \vee v_2 \vee \neg v_3) \wedge (v_1 \vee \neg v_2 \vee \neg v_3) \wedge (\neg v_1 \vee v_2 \vee v_3)$$

It is typically stored in a file named xxx.dimacs, with the following contents:

```
c example DIMACS-CNF 3-SAT
p cnf 3 5
-1 -2 -3 0
1 -2 3 0
1 2 -3 0
1 -2 -3 0
-1 2 3 0
```

We can use Qiskit's built-in PhaseOracle function to construct a circuit for it (the oracle), and then apply amplitude amplification to search for its solution.

The code for circuit drawing is as follows:

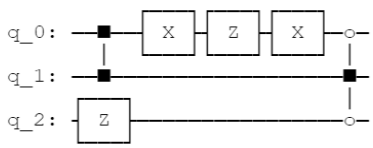
```python
import numpy as np
from qiskit import Aer
from qiskit.visualization import plot_histogram
from qiskit.utils import QuantumInstance
from qiskit.algorithms import Grover, AmplificationProblem
from qiskit.circuit.library import PhaseOracle


class Verifier():
    """Create an object that can be used to check whether
    an assignment satisfies a DIMACS file.
        Args:
            dimacs_file (str): path to the DIMACS file
    """
    def __init__(self, dimacs_file):
        with open(dimacs_file, 'r') as f:
            self.dimacs = f.read()

    def is_correct(self, guess):
        """Verifies a SAT solution against this object's
        DIMACS file.
            Args:
                guess (str): Assignment to be verified.
                             Must be string of 1s and 0s.
            Returns:
                bool: True if `guess` satisfies the
                      problem. False otherwise.
        """
        # Convert characters to bools & reverse
        guess = [bool(int(x)) for x in guess][::-1]
        for line in self.dimacs.split('\n'):
            line = line.strip(' 0')
            clause_eval = False
            for literal in line.split(' '):
                if literal in ['p', 'c']:
                    # line is not a clause
                    clause_eval = True
                    break
                if '-' in literal:
                    literal = literal.strip('-')
                    lit_eval = not guess[int(literal)-1]
                else:
                    lit_eval = guess[int(literal)-1]
                clause_eval |= lit_eval
            if clause_eval is False:
                return False
        return True


with open('test.dimacs', 'r') as f:
```

```
    dimacs = f.read()
print(dimacs)

oracle = PhaseOracle.from_dimacs_file('test.dimacs')
oracle.draw()
```

The output circuit is as follows:

```
c example DIMACS-CNF 3-SAT
p cnf 3 5
-1 -2 -3 0
1 -2 3 0
1 2 -3 0
1 -2 -3 0
-1 2 3 0
```

Out[10]:



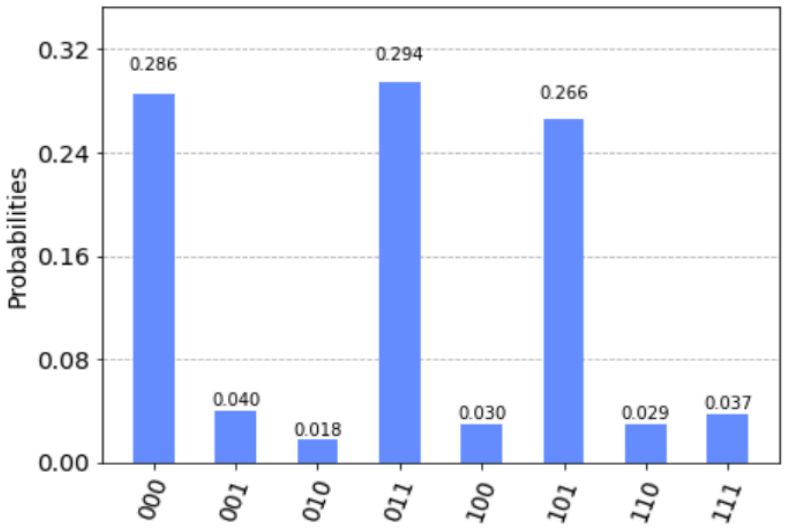
Then, simply apply amplitude amplification. The code is as follows:

```
# Configure backend
backend = Aer.get_backend('aer_simulator')
quantum_instance = QuantumInstance(backend, shots=1024)

# Create a new problem from the phase oracle and the
# verification function
problem = AmplificationProblem(oracle=oracle, is_good_state=v.is_correct)

# Use Grover's algorithm to solve the problem
grover = Grover(quantum_instance=quantum_instance)
result = grover.amplify(problem)
print (result.top_measurement)

plot_histogram(result.circuit_results)
```

The result is shown below; evidently, there are three solutions:

011

Out[12]:


0.32
0.24
0.16
0.08
0.00
Probabilities
0.286
0.040
0.018
0.294
0.030
0.266
0.029
0.037
000
001
010
011
100
101
110
111